

Alberta Buck – Direct Embodiment

Perry Kundert

2026-05-15

1 Why a Second Embodiment

The shipped `BuckKController` measures BUCK against its commodity basket *in dollars*. Each leg – the BUCK/USDT pool, the four basket pools (XAUT/USDT, PAXG/USDC, cbBTC/USDC, WBTC/USDT) – quotes its asset in a USD stablecoin. The PID error is BUCK’s dollar price minus the basket’s dollar cost.

This works. But it has one structural flaw: when the USD inflates, both legs move together. Gold-in-USD rises, BUCK-in-USD rises proportionally, the error stays near zero, `BUCK_K` barely twitches – and the system has just accepted USD inflation as if it were a real signal. The USD denomination has snuck a *reference-currency* into a system that’s supposed to *be* the reference currency.

The direct embodiment fixes this by removing the dollar.

USD-intermediated (legacy):

```
XAUT --[USDT]-- ...
      |
      +--> basket cost in USD ----.
                                   \
                                   error = BUCK$ - basket$ --> PID
                                   /
BUCK --[USDT]-----> BUCK price in USD -----'
```

Direct embodiment:

```
PAXG --[BUCK]-----.
cbBTC --[BUCK]-----+--> sum(amount_i * price_i_in_BUCK) ----.
XAUT --[BUCK]-----'
                                   \
                                   error = 1.0 - basketValue --> PID
```

(no external dollar reference; BUCK measures itself against the basket directly)

The direct embodiment also adds a *new mint path*. Instead of borrowing BUCK against a depreciating insured asset (the `BuckCredit` path), anyone can hand the system a real-world-asset token – PAXG, cbBTC – and receive freshly-minted BUCK at the pool’s current rate, with a receipt NFT that lets them redeem the underlying TOKEN later. The system itself custodies the resulting liquidity position; it never issues a Uniswap V3 NFT to the user.

2 What Changes, What Stays

2.1 Stays the same

- `Buck.sol` as the ERC-20 BUCK with identity binding, demurrage, Jubilee, and the existing `BuckCredit`-backed mint path
- `BuckCredit.sol` unchanged
- `BuckKController` (USD-intermediated) remains shippable for operators who want the alternative credit channel; both controllers conform to the same `IBuckK` surface and are interchangeable from `Buck.sol`'s point of view

2.2 New

- `BuckKControllerBase.sol` – abstract PID core extracted from the existing controller; handles P/I/D state, anti-windup, dT clamp, constructor priming, gain governance. Subclasses override `_readReferences()` and (optionally) `_commitReferences()`.
- `BuckKControllerDirect.sol` – thin subclass. Setpoint = 1.0 (1 BUCK = 1 basket by definition); process = sum of TOKEN/BUCK pool prices weighted by their basket amounts. Reads basket state from `BuckBasket`.
- `BuckBasket.sol` – the system's basket registry and direct-mint orchestrator. Holds the V3 pool addresses, per-deposit bookkeeping, and (custodied) the BUCK side of every depositor's liquidity. Issues `BuckBasketReceipt` NFTs as user-facing claims.
- `BuckBasketReceipt.sol` – bespoke ERC-721 representing a depositor's pending claim on TOKEN held inside `BuckBasket`'s pool position.

2.3 Modified

- `Buck.sol` gets two new entry points: `mintFromBasket(to, amount)` and `burnFromBasket(amount)`. Both gated on `msg.sender == basket` (a single trusted address fixed at construction). These bypass the `BuckCredit` and funding-factor machinery – direct mints have no insurance backing because the BUCK is backed directly by the pool's TOKEN reserves.

3 Direct Mint Lifecycle

Walk a single depositor through the system.

1. Bootstrap (governance):

```
addBasketToken(PAXG, decimals=18, initialPriceInBUCK=4000e18, weight=10000bp)
- V3 pool PAXG/BUCK created at sqrtPriceX96 corresponding to $4000/BUCK
- Constituent recorded: basketAmount = 1.0 / 4000 = 0.00025 PAXG
- PID controller is re-primed against the new process variable
- Pool has zero liquidity; slot0() returns the init price; TWAP not yet meaningful
```

```
addBasketToken(cbBTC, decimals=8, initialPriceInBUCK=100000e18, weight=5000bp)
- PAXG's basketAmount diluted by (1 - 0.5) -> 0.000125 PAXG
```

- cbBTC's basketAmount = $0.5 / 100000 = 5e-6$ cbBTC
- Total basket value at init prices: $0.000125 * 4000 + 5e-6 * 100000 = 1.0$ BUCK
- Pool initialized empty at \$100000/BUCK
- PID re-primed

2. Alice deposits 1 PAXG:

```
alice.depositToken(PAXG, 1e18, maxDeviationBp=50)
- Pool spot read from slot0: 4000 BUCK / PAXG (still init price, no
  liquidity yet)
- TWAP would be 4000 too if observable; deviation guard passes
- Buck.mintFromBasket(BuckBasket, 4000e18) -- 4000 BUCK minted to basket
- BuckBasket calls pool.mint() over the full range with 1 PAXG + 4000 BUCK
- Liquidity L returned, recorded as Alice's share
- Receipt NFT #1 minted to Alice; stores (token=PAXG, principalToken=1e18,
  principalBuck=4000e18, liquidityShare=L)
```

3. External swappers trade through PAXG/BUCK over the next month. Pool collects fees. Spot drifts.

4. Alice redeems receipt #1:

```
alice.redeem(receiptId=1, maxDeviationBp=50)
- Decrease liquidity by L (her full share) -> receives (T_out, B_out)
  where (T_out, B_out) = principal * (1 + accrued_fees_pct) + spot-drift effects
- profit_T = max(0, T_out - 1e18)
- profit_B = max(0, B_out - 4000e18)
- Burn min(B_out, 4000e18) BUCK (principal side)
- Send to Alice: 1e18 PAXG + (profit_T / 2) PAXG + (profit_B / 2) BUCK
- Increase liquidity in the SAME V3 position with (profit_T / 2 PAXG +
  profit_B / 2 BUCK) -- BuckBasket retains this as treasury
- Receipt NFT #1 burned
```

The mechanically important pieces:

- *Pool prices are read at the time of each operation.* Deposit rate is the pool's current spot, not a contract-set price. A depositor who arrives when others have driven the pool away from parity gets a correspondingly skewed mint rate, which is exactly the right behavior – the pool is the price oracle.
- *Liquidity is in BUCK terms, not USD terms.* V3 measures L in pool units; we don't need to translate through any external price.
- *The depositor's principal in BUCK is burned on redeem.* The system's BUCK supply contracts by exactly the amount minted at deposit time, regardless of intervening price drift.
- *Profit splits 50/50.* Half to the agent (in kind: TOKEN + BUCK), half stays in the pool as Buck-owned liquidity. The retained half-share funds future basket rebalancing (v2).

- *Loss eats principal.* If the pool’s BUCK side has been drained by external traders, `B_out < principal_B`, and the burn shrinks by the shortfall. No emergency mint; the PID picks up the slack.

4 Why Custodied Pool Positions

Two reasons the LP token cannot ride free in the depositor’s wallet:

1. *Burn authority.* The principal BUCK must be burned on redeem. Only the contract that controls the liquidity can decrease it and route the BUCK side to `Buck.burnFromBasket`. If the user held the V3 NFT they could withdraw the BUCK and refuse to burn it.
2. *Buck’s profit share.* Half of the LP profit stays in the pool as Buck-controlled liquidity, even after the agent walks away. Buck needs ownership authority to drive future rebalancing operations against that liquidity.

So the V3 position NFT never leaves `BuckBasket`. The depositor’s `BuckBasketReceipt` NFT is a *claim* on a slice of that liquidity, with the principal amounts pinned at deposit time.

5 Sign Convention Stays Canonical

Across both controllers:

```
error = buckValue - basketValue
```

In the legacy USD embodiment, `buckValue` is the BUCK/USDT TWAP and `basketValue` is the commodity basket cost in USD. In the direct embodiment, `buckValue` is literally `1e18` (\$1.00) and `basketValue` is `sum(basketAmount_i * tokenPrice_in_BUCK_i)`.

In both cases, when BUCK is undervalued vs. basket (`basketValue > buckValue`), `error < 0` and positive `Kp` contracts `buckK`. When BUCK is overvalued, the converse. The same two canary tests (`test_sign_convention_*`) cover both controllers via their respective harnesses.

6 PID Priming Across the Refactor

The constructor-time I-priming shipped for `BuckKController` moves into the base class. Subclasses get steady-state continuity for free: deploy at any `initialBuckK` within bounds, the first `compute()` cycle with the system at equilibrium reproduces `initialBuckK`.

The direct embodiment adds one new wrinkle: `addBasketToken` moves the process variable discontinuously (existing constituents get diluted to make room for the new weight). `BuckBasket` calls a privileged `reprime()` on the controller after each successful add, which recaptures `P` and re-derives `I` against the new error. No single-cycle spike, no `I` windup from the discontinuity.

7 Funding Factor in Direct Mode

The insurance funding factor formula stays:

```
factor = max(0, 1 + 10 * (basketValue - buckValue) / basketValue)
```

Just as in the legacy embodiment:

- BUCK undervalued (`basketValue > buckValue`): `factor > 1`, `BuckCredit` mints throttled
- BUCK overvalued (`basketValue < buckValue`): `factor < 1` (down to 0 at +10% deflation), `BuckCredit` mints unconstrained

In direct mode, `buckValue` is the constant 1.0, so the factor collapses to:

```
factor = max(0, 1 + 10 * (basketValue - 1) / basketValue)
```

Equivalent semantics, different denominator.

8 Contract Surface

8.1 `src/BuckKControllerBase.sol` (new, abstract)

```
abstract contract BuckKControllerBase {
    // PID state
    int256 public Kp; int256 public Ki; int256 public Kd;
    int256 public P; int256 public I; int256 public D;

    // References for dS-correction and fundingFactor()
    int256 public lastBuckPrice;
    int256 public lastBasketCost;

    // Output + scheduling
    uint256 public buckK;
    uint256 public dT;
    uint256 public dTMax;
    uint256 public buckKMin;
    uint256 public buckKMax;
    uint256 public lastUpdate;
    bool public primed;

    address public governance;

    int256 constant UNIT = 1e18;

    constructor(int256 _Kp, int256 _Ki, int256 _Kd,
                uint256 _dT,
                uint256 _buckKMin, uint256 _buckKMax, uint256 _buckK,
                address _governance);

    /// Subclass returns the current (buckValue, basketValue). Naming kept
    /// for compatibility with the existing legacy controller's storage
    /// fields; semantics: error = buckValue - basketValue.
    function _readReferences() internal view virtual
        returns (int256 buckValue, int256 basketValue);
```

```

    /// External, permissionless. Runs one PID cycle if dT elapsed.
    function compute() external returns (uint256);

    /// View accessor for the latest cached buckK.
    function currentBuckK() external view returns (uint256);

    /// Counter-cyclical pre-mint reserve requirement.
    function fundingFactor() external view returns (uint256);

    /// Governance setters (Kp/Ki/Kd, dT, dTMax).
    /// ...

    /// Internal helper: re-capture P and I against current references.
    /// Subclasses can call this from privileged paths (e.g. addBasketToken).
    function _reprime() internal;
}

```

8.2 src/BuckKController.sol (modified)

Refactor to extend BuckKControllerBase. Existing oracle reading machinery (`_getBuckPrice` / `_getBasketCost` / Chainlink + V3 basket arrays) stays. Add:

```

function _readReferences() internal view override
    returns (int256 buckValue, int256 basketValue)
{
    buckValue    = _getBuckPrice();
    basketValue = _getBasketCost();
}

```

No behavior change. Existing tests pass.

8.3 src/BuckKControllerDirect.sol (new)

```

interface IBuckBasketRef {
    function basketValueInBuck() external view returns (int256);
}

contract BuckKControllerDirect is BuckKControllerBase {
    IBuckBasketRef public basket;

    constructor(/* PID args + governance */);

    /// One-shot wiring from governance after BuckBasket is deployed.
    function setBasket(address _basket) external;

    function _readReferences() internal view override
        returns (int256 buckValue, int256 basketValue)
    {

```

```

        buckValue    = UNIT;
        basketValue = (address(basket) != address(0))
            ? basket.basketValueInBuck()
            : UNIT;
    }

    /// Privileged hook: BuckBasket calls this after addBasketToken so
    /// the dilution-induced jump in basketValue doesn't manifest as a
    /// single-cycle P/I spike.
    function reprime() external {
        require(msg.sender == address(basket), "only basket");
        _reprime();
    }
}

```

8.4 src/BuckBasket.sol (new)

Holds the entire basket registry and direct-mint orchestrator.

```

contract BuckBasket {
    struct Constituent {
        address token;
        uint8  decimals;
        uint256 basketAmount;          // 18-dec; immutable once added
        uint256 initialPriceInBuck;    // 18-dec; for record-keeping only
        uint24  feeTier;
        address pool;                  // Uniswap V3 pool
    }

    struct Deposit {
        address token;
        uint128 principalToken;        // native decimals
        uint128 principalBuck;         // 18-dec
        uint128 liquidityShare;        // V3 L units
        uint64  depositTime;
    }

    Buck          public buck;
    BuckBasketReceipt public receipt;
    address public controller;
    address public governance;

    Constituent[] public constituents;
    mapping(address => uint256) public indexOf;    // token -> 1+index
    mapping(uint256 => Deposit) public deposits;    // receiptId -> deposit

    uint24 public defaultFeeTier;
    uint32 public twapWindow;
}

```

```

    /// Total liquidity owned by BuckBasket in each pool, including
    /// pending depositor claims AND Buck's retained profit residuals.
    /// Decreased on redeem (depositor's half-profit) and on rebalance.
    mapping(address => uint128) public poolLiquidity;

    // -- Configuration ----- //

    function addBasketToken(
        address token,
        uint8 decimals,
        uint256 initialPriceInBuck,
        uint256 weightBp,
        uint24 feeTier
    ) external; // governance only

    // -- Process variable (called by controller) ----- //

    function basketValueInBuck() external view returns (int256);

    // -- Direct mint / redeem ----- //

    function depositToken(
        address token,
        uint256 tokenAmount,
        uint256 maxDeviationBp
    ) external returns (uint256 receiptId);

    function redeem(
        uint256 receiptId,
        uint256 maxDeviationBp
    ) external;

    // -- Slippage guard helpers ----- //

    function _spotPrice(Constituent storage c) internal view returns (uint256);
    function _twapPrice(Constituent storage c) internal view returns (uint256);
}

```

8.5 src/BuckBasketReceipt.sol (new)

A minimal ERC-721 issued by BuckBasket. Each token represents a single pending deposit claim. Mint and burn are restricted to BuckBasket.

```

contract BuckBasketReceipt is ERC721 {
    address public immutable basket;

    constructor(address _basket) ERC721("BuckBasket Receipt", "BBR");
}

```

```

    function mint(address to) external returns (uint256 id);
    function burn(uint256 id) external;
}

```

The receipt holds no payload beyond identity. Per-deposit data lives in `BuckBasket.deposits[id]`. Transferring the receipt transfers the claim; the new holder calls `BuckBasket.redeem` with the new owner's address verified via `ownerOf(id) = msg.sender`.

8.6 src/Buck.sol (modified)

Add two privileged entry points:

```
address public immutable basket; // wired at construction; address(0) disables direct path
```

```

function mintFromBasket(address to, uint256 amount) external {
    require(msg.sender == basket, "only basket");
    _accrueJubilee();
    _crystallize(to);
    _addBalance(to, amount);
    _totalSupply += amount;
    emit Transfer(address(0), to, amount);
}

```

```

function burnFromBasket(uint256 amount) external {
    require(msg.sender == basket, "only basket");
    _accrueJubilee();
    _crystallize(msg.sender);
    require(_state[msg.sender].balance.asUint() >= amount, "BUCK: insufficient");
    _subBalance(msg.sender, amount);
    _totalSupply -= amount;
    emit Transfer(msg.sender, address(0), amount);
}

```

Neither path consults `buckK` or the funding factor. Direct-mint BUCK is fungible with credit-NFT-mint BUCK; both contribute to `totalSupply` and accrue Jubilee at the system rate.

9 Governance Surface

- `addBasketToken(token, decimals, initialPriceInBuck, weightBp, feeTier)` – only governance. Dilutes existing constituents to maintain 100% weight. Initializes V3 pool at the declared price. Calls `controller.reprime()`.
- `setGains(Kp, Ki, Kd)` – on the controller; standard.
- `setDT, setDTMax` – on the controller; standard.
- `setBasket(address)` – one-shot wiring from governance after `BuckBasket` is deployed. Locks the controller-basket pairing.

Not in v1: `removeBasketToken, setBasketAmount, setFeeTier, rebalance`. All deferred to v2.

10 Failure Modes

10.1 Empty pool slippage guard

When a pool has fresh liquidity (`cardinality=1`, just slot0 obs), TWAP read either reverts or returns the init price. `depositToken` catches the revert and skips the guard for the first deposit only. Subsequent deposits enforce normally.

10.2 Adverse pool drift at redemption

If external traders have drained the pool's BUCK side, `B_out < principal_B`. We burn `min(B_out, principal_B)`; the missing principal stays unbacked in circulation; the PID detects it via `basketValue > 1.0` and contracts credit. The depositor still gets `T_out` TOKEN (more than `principal_T`, since BUCK appreciated against TOKEN in the pool); their loss is the BUCK they didn't redeem.

10.3 Pool dust / minimum-liquidity edge cases

First deposit into an empty pool must be large enough to seed meaningful liquidity (V3 enforces no positive minimum, but sub-1e3 L is fragile). `BuckBasket` reverts if the computed L for the first deposit is below a configurable `minSeedLiquidity` floor.

11 Test Plan

11.1 Layer 1: BuckKControllerBase unit tests

Test the abstract PID core via a minimal test harness that exposes `setReferences(buckValue, basketValue)` and overrides `_readReferences` to return them. Cover:

- Constructor pre-loads I correctly for initial `buckK != UNIT`
- At parity (`buckValue == basketValue` held constant), `compute()` returns initial `buckK`
- Inflation (`buckValue < basketValue`, `error < 0`): `buckK` contracts monotonically (sign convention canary)
- Deflation (`buckValue > basketValue`, `error > 0`): `buckK` expands monotonically (sign convention canary)
- Anti-windup ceiling and floor saturation
- `dTMax` clamp on long-gap cycles
- `reprime()` re-captures P and re-derives I

11.2 Layer 2: BuckKController (legacy) regression

Existing tests (`BuckKControllerUnitTest`, `BuckKControllerV3Test`, `BuckKControllerV3TwapTest`) must continue to pass unchanged after the base-class refactor. No new tests; the refactor is mechanical.

11.3 Layer 3: BuckKControllerDirect unit tests

Mock `BuckBasket` returns configurable `basketValueInBuck()`. Deploy `BuckKControllerDirect`, `setBasket` to the mock. Cover:

- Pre-`setBasket`: `_readReferences` returns (UNIT, UNIT); error = 0
- After `setBasket`: process variable tracks the mock
- Same two sign-convention canaries as Layer 1, this time against a real direct controller
- `reprime()` callable only by basket; absorbs a step in `basketValueInBuck()` without I or P spike
- `fundingFactor()` produces the canonical counter-cyclical output in direct units

11.4 Layer 4: BuckBasket unit tests (no real V3)

Mock the V3 pool to return a configurable spot tick and a configurable TWAP. Mock `Buck.mintFromBasket` / `burnFromBasket` as simple internal balance tracking. Cover:

- `addBasketToken` records the constituent and dilutes existing weights so they sum to 100% (`basketAmount * initialPrice` sum = 1.0 at init)
- Second `addBasketToken` further dilutes; first's new `basketAmount` = old * (1 - new_weight)
- `depositToken` mints the correct amount of BUCK against the current pool spot
- `depositToken` reverts when $|\text{spot} - \text{TWAP}| / \text{TWAP} > \text{maxDevBp}$
- `depositToken` skips the deviation guard when TWAP read reverts (cold-pool case)
- `redeem` burns $\min(B_{\text{out}}, \text{principal}_B)$ regardless of profit sign
- `redeem` sends `principal_T + half_profit_T` TOKEN to depositor
- `redeem` retains `half_profit_T` TOKEN + `half_profit_B` BUCK inside `BuckBasket`
- `redeem` burns receipt NFT
- `redeem` calls `controller.compute()`

11.5 Layer 5: end-to-end integration

Deploy the full stack on a Forge fixture: Identity, BuckCredit, Buck, BuckKControllerDirect, BuckBasket, BuckBasketReceipt, plus real Uniswap V3 factory/pools. Scenario:

1. Governance `addBasketToken(PAXG, ..., 5000bp, 500)` (PAXG @ \$4000/BUCK, half-basket weight, 0.05% fee tier)
2. Governance `addBasketToken(cbBTC, ..., 5000bp, 500)` (cbBTC @ \$100000/BUCK, half-basket weight)
3. Alice deposits 1 PAXG and 0.04 cbBTC across two transactions

4. External actors run swaps through both pools, accruing fees and drifting spot
5. `controller.compute()` reports a sensible PID trajectory
6. Alice redeems both receipts; receives principal + half-profit; principal BUCK burned
7. Verify `totalSupply` equals exactly the residuals (Buck's half-profit BUCK)
8. Verify the residual liquidity remains inside the pools (Buck's retained share)

11.6 Layer 6: equilibrium simulation (analog of BuckKArbScenarioTest)

Recreate the existing 18-month Carol/Hank simulation against the direct embodiment. Two basket constituents (gold-pegged + BTC-pegged synthetic tokens, both 6-dec mocks), starting weights 50/50. Confirm the same equilibrium-seeking behavior (BUCK price oscillates around basket-implied parity, `fundingFactor` responds counter-cyclically) without the USD reference.

11.7 Acceptance gate

All six layers green plus the existing 277 non-fork tests for the legacy embodiment. Final commit message should list the new contracts, the test count delta, and the gas-snapshot delta for any modified existing contracts.

12 Out of Scope for v1

- *Basket rebalancing.* Buck retains profit residuals as treasury liquidity but does not redeploy them; v2 adds `BuckBasket.rebalance()`.
- *Fee tier migration.* V3 pool fees are immutable; changing a constituent's fee tier requires deploying a new pool and migrating liquidity. Defer.
- *Basket constituent removal.* Once added, a token stays. Requires burning all corresponding BUCK first.
- *Concentrated liquidity ranges.* All `BuckBasket` positions are full-range (`MIN_TICK` .. `MAX_TICK`).
- *Cross-pool atomic swaps.* External users use Uniswap V3 directly for PAXG -> BUCK -> cbBTC; `BuckBasket` doesn't intermediate.